

1901 软件应用手册

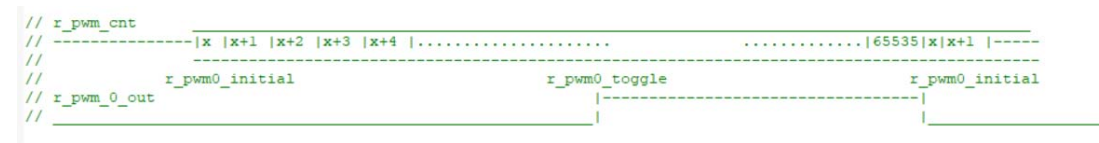
为了方便工程师和 AI（VS Code）能更好的阅读和理解代码，我们写了一个 API 手册，如果你是 AI 编写代码，请把这个文档“喂”给 AI，它能更好更快的理解 BSR1910 的 SDK 底层代码。如果有未实现的 API 函数，请告诉我们的工程师，我们会现场帮助实现底层 API 代码。

目录

一, PWM 接口	2
1.1, hw_pwm_enable(HW_PWM_CHAN_T channel)	4
1.2, void hw_pwm_disable(HW_PWM_CHAN_T channel)	5
1.3, void hw_pwm_invert(HW_PWM_CHAN_T channel)	5
1.4, void Config_PWM(HW_PWM_CHAN_T channel,const struct HAL_PWM_CFG_T *cfg)	5
二, SPI 接口	6
2.1, void HW_SPI_Initialise(HAL_SPI_ID_T id)	7
2.2, void SPI_8bit_Transfer(void)	8
2.3, void SPI_16bit_Transfer(void)	8
2.4, void SPI_24bit_Transfer(void)	8
2.5, void SPI_32bit_Transfer(void)	8
2.6, void HW_SPI_Tx_DMA_8bit(HAL_SPI_ID_T id,uint16 *pData, uint16 DataLen)	8
2.7, void HW_SPI_Tx_DMA_16bit_ColorBlock(HAL_SPI_ID_T id,uint16 *pData, uint16 DataLen)	9
2.8, void HW_SPI_Tx_DMA_32bit(HAL_SPI_ID_T id,uint16 *pData, uint16 DataLen)	9
三, GPIO 接口	9
3.1, void GpiopinMode(uint8 pin,uint8 mode)	10
3.2, void GpiopinWrite(uint8 pin,uint8 val)	10
3.3, uint8 GpiopinRead(uint8 pin)	10
四, UART 接口	11
4.1, void Gecko_Uart0_Send_Char(char s)	12
4.2, void Gecko_Uart1_Send_Char(char s)	12
4.3, void Uart_16550_Initialise(HAL_UART_ID_T id,uint32 baud_rate, uint16 uart_setting)	12
五, DMA 接口	13
5.1, void DMA_Configuration(void)	14
5.2, __RAM_CODE__ void Clear_DMA_Interrupt(void)	14
六, GUI 接口	14
6.1, void Lcd_SetRegion(uint16_t x_start,uint16_t y_start,uint16_t x_end,uint16_t y_end)	14
6.2, void Lcd_Clear(uint16_t Color)	15
6.3, void Lcd_Fill(uint16_t x,uint16_t y,uint16_t xend,uint16_t yend,uint16_t Color)	15
6.4, void sc_draw_Fill(sc_pfb_t *dest, int xs, int ys, int w, int h, color_t color, uint16_t alpha);	15
6.5, void sc_draw_Text(sc_pfb_t *dest, int tx, int ty, lv_font_t *font, const char *text, color_t tc, color_t bc);	16
6.6, void sc_draw_Num(sc_pfb_t *dest, int tx, int ty, int w, int h, lv_font_t *font, int num, int den, color_t tc, color_t bc);	16
6.7, void Gui_DrawPoint(uint16_t x,uint16_t y,uint16_t Data)	16

一, PWM 接口

1901 包含 8 个 16bit 的 PWM 模块。每个 PWM 都支持反向，死区控制。



PWM 模块基地址

```
//AHB Address
```

```
#define GECKO_PWM_BASE_NEW 0x40005000
```

```
typedef volatile struct
```

REG32	pwm_select_en;	//0x00000000
REG32	r_pwm01_initial;	//0x00000004
REG32	r_pwm23_initial;	//0x00000008
REG32	r_pwm01_toggle;	//0x0000000C
REG32	r_pwm23_toggle;	//0x00000010
REG32	r_pwm01_phase;	//0x00000014
REG32	r_pwm23_phase;	//0x00000018
REG32	r_pwm45_initial;	//0x0000001C
REG32	r_pwm45_toggle;	//0x00000020
REG32	r_pwm45_phase;	//0x00000024
REG32	r_pwm03_Td_cycle;	//0x00000028
REG32	r_pwm47_Td_cycle;	//0x0000002C
REG32	r_pwm67_initial;	//0x00000030
REG32	r_pwm67_toggle;	//0x00000034
REG32	r_pwm67_phase;	//0x00000038
REG32	pwm_reg0x3c;	//0x0000003C
REG32	pwm_reg0x40;	//0x00000040
REG32	pwm_reg0x44;	//0x00000044
REG32	pwm_reg0x48;	//0x00000048
REG32	pwm_reg0x4c;	//0x0000004C
REG32	pwm_reg0x50;	//0x00000050

```
} Gecko_PWM_T;
```

```
#define Gecko_PWM ((Gecko_PWM_T*) GECKO_PWM_BASE_NEW)
```

PWM 寄存器表:

REG_00		PWM enable control register	
bit num			
<0>	PWM_CH0_EN	pwm channel 0 enable	1'b0
<1>	PWM_CH1_EN	pwm channel 1 enable	1'b0
<2>	PWM_CH2_EN	pwm channel 2 enable	1'b0
<3>	PWM_CH3_EN	pwm channel 3 enable	1'b0
<4>	PWM_CH4_EN	pwm channel 4 enable	1'b0
<5>	PWM_CH5_EN	pwm channel 5 enable	1'b0
<6>	PWM_CH6_EN	pwm channel 6 enable	1'b0
<7>	PWM_CH7_EN	pwm channel 7 enable	1'b0
<8>	PWM_CH0_INV	pwm channel 0 invert	1'b0
<9>	PWM_CH1_INV	pwm channel 1 invert	1'b0
<10>	PWM_CH2_INV	pwm channel 2 invert	1'b0
<11>	PWM_CH3_INV	pwm channel 3 invert	1'b0
<12>	PWM_CH4_INV	pwm channel 4 invert	1'b0
<13>	PWM_CH5_INV	pwm channel 5 invert	1'b0
<14>	PWM_CH6_INV	pwm channel 6 invert	1'b0
<15>	PWM_CH7_INV	pwm channel 7 invert	1'b0

REG_04		pwm channel0 and channel1 initial	
bit num			
<15:0>	PWM0_Initial_Value		16'b0
<31:16>	PWM1_Initial_Value		16'b0

REG_08		pwm channel2 and channel3 initial	
bit num			
<15:0>	PWM2_Initial_Value		16'b0
<31:16>	PWM3_Initial_Value		16'b0

REG_0C		pwm channel0 and channel1 toggle value	
bit num			
<15:0>	PWM0_toggle_Value		16'b0
<31:16>	PWM1_toggle_Value		16'b0

REG_10		pwm channel2 and channel3 toggle value	
bit num			
<15:0>	PWM2_toggle_Value		16'b0
<31:16>	PWM3_toggle_Value		16'b0

REG_1C		pwm channel4 and channel5 initial	
bit num			
<15:0>	PWM4_Initial_Value		16'b0
<31:16>	PWM5_Initial_Value		16'b0

REG_20		pwm channel4 and channel5 toggle value	
bit num			
<15:0>	PWM4_toggle_Value		16'b0
<31:16>	PWM5_toggle_Value		16'b0

REG_28		pwm channel0,1,2,3dead zone control value	
bit num			
<7:0>	PWM0_Td_Value	dead zone ctrl time	4'b0
<15:8>	PWM1_Td_Value	dead zone ctrl time	4'b0
<23:16>	PWM2_Td_Value	dead zone ctrl time	4'b0
<31:24>	PWM3_Td_Value	dead zone ctrl time	4'b0

REG_2C		pwm channel4,5,6,7 dead zone control value	
bit num			
<7:0>	PWM4_Td_Value	dead zone ctrl time	4'b0
<15:8>	PWM5_Td_Value	dead zone ctrl time	4'b0
<23:16>	PWM6_Td_Value	dead zone ctrl time	4'b0
<31:24>	PWM7_Td_Value	dead zone ctrl time	4'b0

REG_30		pwm channel6 and channel7 Initial value	
bit num			
<15:0>	PWM6_Initial_Value		16'b0
<31:16>	PWM7_Initial_Value		16'b0

REG_34		pwm channel6 and channel7 toggle value	
bit num			
<15:0>	PWM6_toggle_Value		16'b0
<31:16>	PWM7_toggle_Value		16'b0

1.1, hw_pwm_enable(HW_PWM_CHAN_T channel)

PWM 通道使能，参数 channel。

用法：

```
hw_pwm_enable(HW_PWM_CHAN_3);
```

1.2, void hw_pwm_disable(HW_PWM_CHAN_T channel)

PWM 通道输出关闭，参数 channel。

用法：

```
hw_pwm_disable(HW_PWM_CHAN_3);
```

1.3, void hw_pwm_invert(HW_PWM_CHAN_T channel)

PWM 通道反向输出，参数 channel。

用法：

```
hw_pwm_invert(HW_PWM_CHAN_7);
```

1.4, void Config_PWM(HW_PWM_CHAN_T channel,const struct HAL_PWM_CFG_T *cfg)

```
struct HAL_PWM_CFG_T {  
    uint32_t freq; //expected frequency  
    uint8_t ratio; //expected duty  
    uint8_t Tdead_cycle_count;  
};
```

配置 PWM 通道的参数，有 2 个参数：

参数 1，channel

参数 2，*cfg

用法：

```
SixChCfg.freq= 200000;//配置 PWM 频率
```

```
SixChCfg.ratio= 50;//配置 PWM 占空比
```

```
SixChCfg.Tdead_cycle_count=1;//配置 PWM 死区的 cycle
```

```
Config_PWM(HW_PWM_CHAN_1,&SixChCfg);
```

二, SPI 接口

BSR1901 的 SPI 接口可以认为是专门用于 LCD 显示屏的, SPI 的通信速率可以达到 25Mbit/s, 可以驱动 65536 色的 LCD。这个接口有两个片选信号, 支持双屏异显。

SPI 特点:

- 1, 3 线全双工同步传输
- 2, 8 位, 16 位, 24 位, 32 位传输帧选择
- 3, 支持 DMA 传输
- 4, 可编程的时钟极性和相位
- 5, 可编程的数据顺序, MSB 在前或者 LSB 在前
- 6, 可触发中断

SPI 分类

Standard SPI (标准SPI)

全双工

标准SPI, 通信线包含: 片选CS、时钟线CLK、输入DI、输出DO。驱动SPI FLASH还需要写保护WP以及维持HOLD引脚。

Dual SPI (双线SPI)

半双工

对标准SPI改进, DO和DI改成IO0和IO1, 变为双向IO口。

一个周期内, 通过数据线, 传输2位数据。

操作SPI FLASH:

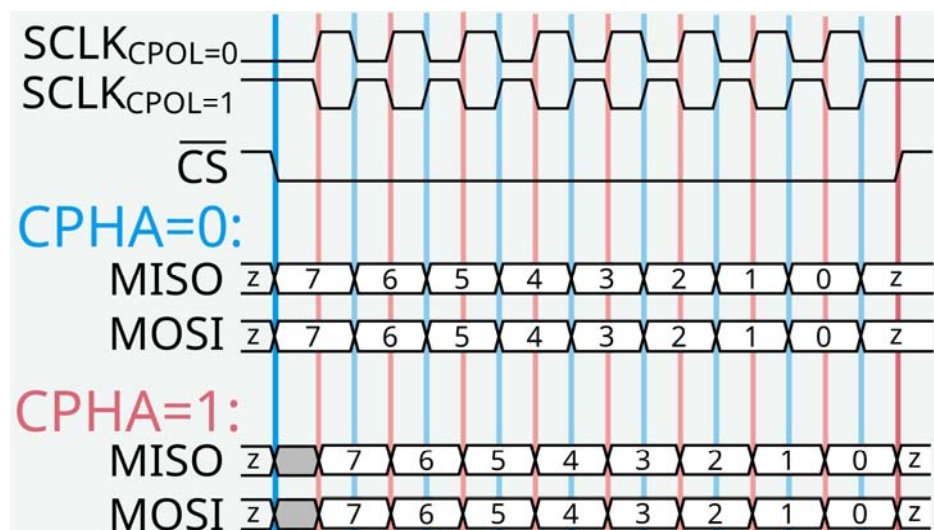
速度至上

Quad SPI (四线SPI)

半双工

对Dual SPI改进, 写保护WP和维持HOLD复用为数据IO口, 为IO2和IO3。

一个周期内, 通过数据线, 传输4位数据。



寄存器描述:

REG_00	SPI_DATA_REG		32'b0
[31:0]	tx_fifo_data/rx_fifo_data		
REG_04	SPI_Control_REG		32'b0
[3:0]	reserved		
[5:4]	size	2'b00:8bits; 2'b01:16bits; 2'b10:32bits; 2'b11:24bits	
[6]	lsbf		
[7]	cpha	clock phase 0:sck toggle at the middle of first data bit 1:sck toggle at the beginning of first data bit	
[8]	cpol	clock polarity 0: idle state sck low 1: idle state sck high	
[15:9]	reserved		
[31:16]	spi_baud		
REG_08	SPI_STATUS_REG		4'b0
[0]	tx_fifo_empty		
[1]	tx_fifo_full		
[2]	rx_fifo_full		
[3]	busy_spi_sync		
REG_0C	SPI_INT_CTL_REG		4'b0
[0]	tx_fifo_empty_int		
[1]	tx_fifo_full_int		
[2]	rx_fifo_full_int		
[3]	busy_spi_fall_pulse		

2.1, void HW_SPI_Initialise(HAL_SPI_ID_T id)

初始化 SPI 接口，参数 id(1901 芯片只保留 SPI_0)

```
typedef enum {
    HAL_SPI_0                = 0x00000000,
    HAL_SPI_1                = 0x00000001, //New SPI--->2022-09-04    HAL_SPI_QTY
} HAL_SPI_ID_T;
```

用法:

```
HW_SPI_Initialise(HAL_SPI_0);
```

2.2, void SPI_8bit_Transfer(void)

设置 SPI 为 8bit 传输，无参数

2.3, void SPI_16bit_Transfer(void)

设置 SPI 为 16bit 传输，无参数

2.4, void SPI_24bit_Transfer(void)

设置 SPI 为 24bit 传输，无参数

2.5, void SPI_32bit_Transfer(void)

设置 SPI 为 32bit 传输，无参数

2.6, void HW_SPI_Tx_DMA_8bit(HAL_SPI_ID_T id,uint16 *pData, uint16 DataLen)

设置 DMA 按照 8bit 读取数据，同时 SPI 按照 8bit 格式传输输出数据，参数:

第一个 id

第二个 pData

第三个 DataLen

用法:

```
HW_SPI_Tx_DMA_8bit(HAL_SPI_0, (uint16*)gImage_128x128_battery, 32768);
```

2.7, void HW_SPI_Tx_DMA_16bit_ColorBlock(HAL_SPI_ID_T id,uint16 *pData, uint16 DataLen)

DMA 在显示屏上刷新色块。

设置 DMA 按照 16bit 读取数据，同时 SPI 按照 16bit 格式传输输出数据，参数：

第一个 id

第二个 pData

第三个 DataLen

用法：

```
uint16_t color = BLACK;
```

```
uint32_t len = X_MAX_PIXEL*Y_MAX_PIXEL;
```

```
HW_SPI_Tx_DMA_16bit_ColorBlock(HAL_SPI_0,&color,len);
```

2.8, void HW_SPI_Tx_DMA_32bit(HAL_SPI_ID_T id,uint16 *pData, uint16 DataLen)

设置 DMA 按照 32bit 读取数据，同时 SPI 按照 32bit 格式传输输出数据，参数：

第一个 id

第二个 pData

第三个 DataLen

用法：

```
HW_SPI_Tx_DMA_32bit(HAL_SPI_0, (uint16*)fire_eye_7, 12800);
```

三, GPIO 接口

1901 一共两组 GPIO，每组 8 个 GPIO 口，分别是 GPIO0 和 GPIO1。

GPIO 模块基地址：0x40018000(GPIO0)和 0x40019000(GPIO1)

```
#define GPIO_GROUP_A_BASE      (0x40018000)
```

```
#define GPIO_GROUP_B_BASE      (0x40019000)
```

```
#define GPIO_GROUP0_BASE       0x40018000
```

```
#define GPIO_GROUP1_BASE       0x40019000
```

3.1, void GpiopinMode(uint8 pin,uint8 mode)

设置 gpio 的输入输出模式，参数 mode

```
enum {  
    LOW = 0x00,HIGH,TOGG,  
    INPUT = 0x00,OUTPUT,s  
    PA0 = 0x00,PA1,PA2,PA3,PA4,PA5,PA6,PA7,  
    PB0 = 0x08,PB1,PB2,PB3,PB4,PB5,PB6,PB7,  
};
```

用法:

```
GpiopinMode(PB3,OUTPUT);//输出模式  
GpiopinWrite(PB3,HIGH);//输出高电平
```

3.2, void GpiopinWrite(uint8 pin,uint8 val)

设置 gpio 的输出高或者低电平，参数 1，pin；参数 2，val

用法:

```
GpiopinWrite(PB3,HIGH);//输出高电平
```

3.3, uint8 GpiopinRead(uint8 pin)

设置 gpio 的状态，参数，pin

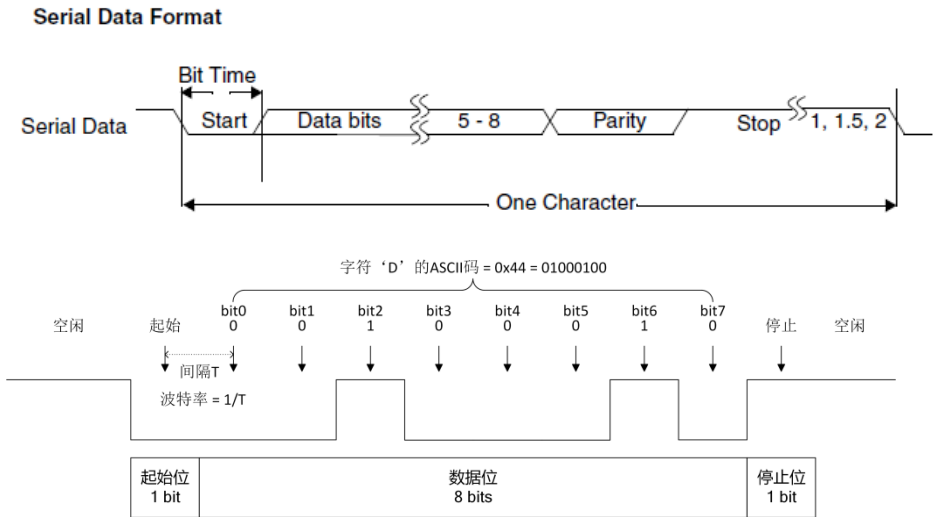
用法:

```
#define KEY1  (PB5)  
//GpioPinRead(KEY1);  
if(GpiopinRead(KEY1) != 0x20)  
{  
    GpiopinMode(PB3,OUTPUT);  
    GpiopinWrite(PB3,HIGH);  
}  
  
//PAD11(GPIOA7)  
gpio_status=K27_KEY_Detect();//default gpio_status=0x80  
key_status=gpio_status & 0x80;
```

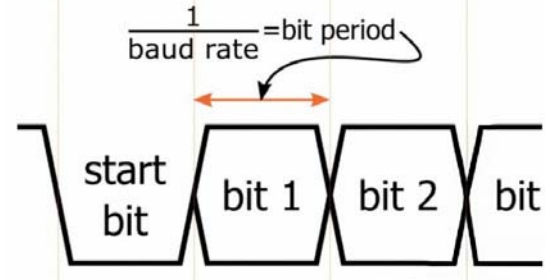
四, UART 接口

1109 有两个 UART 模块。UART 模块基地址: 0x40001000/0x40003000

UART 的数据格式:



常见的有 9600bps、115200bps 等，关于波特率这里说明一下：



在只有起始位和 1 位停止位的情况下，传输一个字节的数据包长度为 10 个 bits，这种情况下的每秒钟可以传输的字节数为：

9600/10=960 bytes。从下图中，我们可以看到在 115200bps 时，一个 bit 的宽度是 8.68us，供客户调试参考。

Bauds	Transmission speed			Real transmission speed	
	Bit/s	Bit duration	Speed	Speed	Byte duration
50 Bd	50 bits/s	20.000 ms	6.25 bytes/s	5 bytes/s	200.000 ms
75 Bd	75 bits/s	13.333 ms	9.375 bytes/s	7.5 bytes/s	133.333 ms
110 Bd	110 bits/s	9.091 ms	13.75 bytes/s	11 bytes/s	90.909 ms
134 Bd	134 bits/s	7.463 ms	16.75 bytes/s	13.4 bytes/s	74.627 ms
150 Bd	150 bits/s	6.667 ms	18.75 bytes/s	15 bytes/s	66.667 ms
200 Bd	200 bits/s	5.000 ms	25 bytes/s	20 bytes/s	50.000 ms
300 Bd	300 bits/s	3.333 ms	37.5 bytes/s	30 bytes/s	33.333 ms
600 Bd	600 bits/s	1.667 ms	75 bytes/s	60 bytes/s	16.667 ms
1200 Bd	1200 bits/s	833.333 μs	150 bytes/s	120 bytes/s	8.333 ms
1800 Bd	1800 bits/s	555.556 μs	225 bytes/s	180 bytes/s	5.556 ms
2400 Bd	2400 bits/s	416.667 μs	300 bytes/s	240 bytes/s	4.167 ms
4800 Bd	4800 bits/s	208.333 μs	600 bytes/s	480 bytes/s	2.083 ms
9600 Bd	9600 bits/s	104.167 μs	1200 bytes/s	960 bytes/s	1.042 ms
19200 Bd	19200 bits/s	52.083 μs	2400 bytes/s	1920 bytes/s	520.833 μs
28800 Bd	28800 bits/s	34.722 μs	3600 bytes/s	2880 bytes/s	347.222 μs
38400 Bd	38400 bits/s	26.042 μs	4800 bytes/s	3840 bytes/s	260.417 μs
57600 Bd	57600 bits/s	17.361 μs	7200 bytes/s	5760 bytes/s	173.611 μs
76800 Bd	76800 bits/s	13.021 μs	9600 bytes/s	7680 bytes/s	130.208 μs
115200 Bd	115200 bits/s	8.681 μs	14400 bytes/s	11520 bytes/s	86.806 μs
230400 Bd	230400 bits/s	4.340 μs	28800 bytes/s	23040 bytes/s	43.403 μs
460800 Bd	460800 bits/s	2.170 μs	57600 bytes/s	46080 bytes/s	21.701 μs
576000 Bd	576000 bits/s	1.736 μs	72000 bytes/s	57600 bytes/s	17.361 μs
921600 Bd	921600 bits/s	1.085 μs	115200 bytes/s	92160 bytes/s	10.851 μs

4.1, void Gecko_Uart0_Send_Char(char s)

UART_0 发送一个字符，参数 s：通常用于打印 log 信息。

用法：

```
/* retarget the C library printf function to the USART */
int fputc(int ch, FILE *f)
{
    Gecko_Uart0_Send_Char(ch);
    return ch;
}
```

4.2, void Gecko_Uart1_Send_Char(char s)

UART_1 发送一个字符，参数 s：通常用于打印 log 信息。

用法：

```
/* retarget the C library printf function to the USART */
int fputc(int ch, FILE *f)
{
    Gecko_Uart1_Send_Char(ch);
    return ch;
}
```

4.3, void Uart_16550_Initialise(HAL_UART_ID_T id,uint32 baud_rate, uint16 uart_setting)

UART 初始化，参数 1，id；参数 2，baud_rate；参数 3，uart_setting

用法：

```
Uart_16550_Initialise(HAL_UART_0,115200,0x3);
Uart_16550_Initialise(HAL_UART_1,115200,0x3);
```

五, DMA 接口

1901 的 DMA 在设计的时候, 优先考虑的应用场景就是 LCD 刷屏。

关于 DMA 寄存器的说明:

```
#define AHB_DMA_CONTROL_REG          (AHB_DMA_BASE + 0x00)
#define AHB_DMA_SRCADDR_REG          (AHB_DMA_BASE + 0x04)
#define AHB_DMA_DESTADDR_REG         (AHB_DMA_BASE + 0x08)
#define AHB_DMA_DATALENGTH_REG       (AHB_DMA_BASE + 0x0C)

//New Register for GEK1109

#define AHB_DMA_DONE_STS_REG          (AHB_DMA_BASE + 0x10) //dma done status register
#define AHB_DMA_STEP_CTRL_REG        (AHB_DMA_BASE + 0x14) //dma carry data step control register
```

DMA 寄存器说明:

REG_00	DMA_CTRL		8'b0
<0>	fix_src		
<1>	fix_des		
<4:2>	hsizem	3'b000:byte; 3'b001:2-byte; Other values:4-byte;	
<5>	reserved		
REG_04	SRC_ADDR		32'b0
<31:0>	source address		
REG_08	DES_ADDR		32'b0
<31:0>	destination address		
REG_0C	DATA_LENGTH		32'b0
<31:0>	length		
REG_10	STS		32'b0
<0>	dma_done		
<1>	dma_int_sts	interrupt status	
<31:2>	reserved		
REG_14	STEP_CTRL		32'b0
<15:0>	hready_cnt_th		
<16>	dma_int_clr		
<17>	dma_int_msk		
<18>	dma_for_dac		
<31:19>	reserved		

```
//register 0x14
#define dma_for_dac_en          (1<<18)
#define dma_int_mask           (1<<17)//set this bit "1" to enable DMA Interrupt.
#define dma_int_clear          (1<<16)
```

5.1, void DMA_Configuration(void)

初始化 DMA，无参数

用法：

在 main 函数中调用

```
DMA_Configuration();
```

5.2, __RAM_CODE__ void Clear_DMA_Interrupt(void)

清除 DMA 中断，无参数。 DMA 每完成一次传输，都会产生一次中断。

用法：

在 ISR 中调用

```
__RAM_CODE__ void DMA_IRQHandler(void)
{
    dma_int_flag = 1;
    Clear_DMA_Interrupt();
}
```

六, GUI 接口

1901 有自己的 GUI，同时集成了 SCGUI，客户可以根据需要处理。

6.1, void Lcd_SetRegion(uint16_t x_start,uint16_t y_start,uint16_t x_end,uint16_t y_end)

设置需要刷新的 LCD 区域，参数是区域的四个坐标

用法：

```
Lcd_SetRegion(0, 0, 127, 127);//128x128 分辨率
```

6.2, void Lcd_Clear(uint16_t Color)

全屏填充颜色，参数 Color

```
void Lcd_Clear(uint16_t Color)
{
    uint32_t len = X_MAX_PIXEL*Y_MAX_PIXEL;
    Lcd_SetRegion(0,0,X_MAX_PIXEL-1,Y_MAX_PIXEL-1);
    HW_SPI_Tx_DMA_16bit_ColorBlock(HAL_SPI_0,&Color,len);
}
```

用法：

```
Lcd_Clear(RED);
```

6.3, void Lcd_Fill(uint16_t x,uint16_t y,uint16_t xend,uint16_t yend,uint16_t Color)

指定区域填充颜色

用法：

```
Lcd_Fill(22, 41, 22+80.0/100*percent, 84, 0x051d);
```

6.4, void sc_draw_Fill(sc_pfb_t *dest, int xs, int ys, int w, int h, color_t color, uint16_t alpha);

在 LCD 上绘制矩形,参数:

- `dest`: 目标 pfb (帧缓冲区), 可为 NULL 表示直接绘制到屏幕
- `xs, ys`: 矩形左上角坐标
- `w, h`: 矩形的宽度和高度【`w, h` 中任意一个设为 1 即可形成单像素宽度的直线】
- `color`: 填充颜色
- `alpha`: 透明度 (0-255)【`alpha` 设为 255 表示不透明】

用法:

```
// 绘制红色实心矩形
```

```
sc_draw_Fill(NULL, 50, 50, 100, 80, C_RED, 255);
```

sc_draw_Fill()可以用来画直线，只需将矩形的宽度或高度设置为 1 像素，即可得到水平或垂直的直线。

例如:

```
/* 水平直线（长度 100 像素，颜色红色） */
```

```
sc_draw_Fill(NULL, 10, 30, 100, 1, C_RED, 255);
```

```
/* 垂直直线（长度 80 像素，颜色蓝色） */
```

```
sc_draw_Fill(NULL, 50, 10, 1, 80, C_BLUE, 255);
```

6.5, void sc_draw_Text(sc_pfb_t *dest, int tx, int ty, lv_font_t *font, const char *text, color_t tc, color_t bc);

在 LCD 上显示简单文本，参数：

参数说明：

- `dest`: PFB 目标缓冲区（NULL 表示直接绘制到屏幕）
- `tx`, `ty`: 文本起始坐标（左上角）
- `font`: 字体指针（如`gui->font`）
- `text`: 要显示的字符串
- `tc`: 文字颜色（如 C_RED, C_BLUE 等）
- `bc`: 背景颜色（通常使用`gui->bkc`）

用法：

```
// 在坐标(10,10)处显示"Hello World", 红色文字，默认背景
sc_draw_Text(NULL, 10, 10, gui->font, "Hello World", C_RED, gui->bkc);
```

```
// 显示变量内容
```

```
char str[20];
sprintf(str, "温度: %d° C", temperature);
sc_draw_Text(NULL, 50, 50, gui->font, str, C_WHITE, gui->bkc);
```

6.6, void sc_draw_Num(sc_pfb_t *dest, int tx, int ty, int w, int h, lv_font_t *font, int num, int den, color_t tc, color_t bc);

在 LCD 上显示数字，参数：

- `w`, `h`: 显示区域的宽高
- `num`: 分子
- `den`: 分母（用于显示小数，如 100 表示保留 2 位小数）

```
// 显示整数 123
```

```
sc_draw_Num(NULL, 100, 30, 40, 20, gui->font, 123, 1, C_BLUE, C_RED);
```

```
// 显示小数 12.34（分子 1234，分母 100）
```

```
sc_draw_Num(NULL, 100, 60, 60, 20, gui->font, 1234, 100, C_GREEN, gui->bkc);
```

6.7, void Gui_DrawPoint(uint16_t x, uint16_t y, uint16_t Data)

画一个点

用法：

```
Gui_DrawPoint(x + j, y + i, BLACK);
```